

birdclef-2026

14th place solo gold solution

Rank	14
Team	Dieter
Author	Dieter
Topic ID	704864
Version	Original

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704864>

Retrieved with Kaggle CLI on 2026-07-03.

BirdCLEF+ 2026 14th Place Solution

Final private LB: **0.95531**, rank **14**.

Short version

The final submission was a three-branch ensemble:

1. **SED main**: 20 s EfficientNetV2-B3 SED student, full 234-class taxonomy, trained with blended soft pseudo labels and sonotype-specific pseudo-label sharpening.
2. **SED small**: the same base recipe, but with a BirdNET-v3-style global/frequency/time neck and a wider hop. This was mostly a diversity branch.
3. **ProtoSSM / Perch branch**: Perch v2 logits and 1536-d embeddings, MLP probes, site/hour priors, a lightweight sequence model, and a residual sequence correction.

The final blend was histogram-matched ProtoSSM predictions to one main SED per class, then used group-specific weights:

group	SED main	SED small	ProtoSSM
Aves	0.72	0.18	0.10
non-Aves	0.40	0.10	0.50

Data and validation

I used the competition train audio plus the labelled train soundscapes. The working fold split was soundscape-aware split with a mix of shared soundscapes and exclusive soundscapes per fold (same in test set).

Validation was useful for catching broken changes and for group-level diagnostics, but not for final model selection. The train soundscape labels were too small to trust absolute CV. Final choices were made from a mix of:

- full-taxonomy validation sanity checks,
- Aves / non-Aves / Insecta / sonotype metric splits,
- public LB movement,
- runtime feasibility,
- and correlation/diversity between branches.

The sonotype columns were the main reason to treat taxa separately. They have no isolated focal recordings and are hard to separate acoustically, so I did not want the final model to be an Aves-only leaderboard optimizer.

SED students

Both SED branches used `tf_efficientnetv2_b3.in21k_ft_in1k`, 32 kHz audio, 20 s clips, 224 mel bins, `n_fft=4096`, `fmax=16000`, BCE-style window and clip losses, EMA, SpecAugment, and OpenVINO export for CPU inference.

`tf_efficientnetv2_b3`

This was the main SED branch.

Important settings:

- 20 s context with four 5 s output windows.
- Full 234-class taxonomy.
- Pretrained on XCL
- Soft pseudo labels from the internal `blend4` teacher.
- Sonotype pseudo-label probabilities power-raised by $1 / 0.65$ before pseudo/train-audio mixup.
- Soundscape/train-audio forced mixup with lambda clamped to $[0.3, 0.7]$.
- `mixup_prob=0.5`, `soundscape_oversample=2`

The useful part was not generic pseudo-labeling. It was pseudo-labeling with a taxon-specific transform. Applying the power only to sonotypes was more stable than globally sharpening every class.

`blend` pseudo-label teacher

`blend` was the internal teacher used for the SED student pseudo labels. It was not one model, and it was not the same as the final submission ensemble. I built it as a group-wise teacher because the best source of signal was different for birds, frogs/reptiles, sonotypes, and the remaining non-Aves classes.

Component	Model specifics	Target used in teacher	Weight in <code>blend</code>	Why it was included
BirdNET V3 Aves specialist	BirdNET V3 11K backbone, 20 s context, Aves-only head, WABAD/PteroSet soundscape rows	Aves	55% Aves, histogram anchor	Strongest and best-calibrated bird teacher; good coverage for common vocal birds.
Perch ProtoSSM Aves specialist	Perch v2 1536-d embeddings, MLP probes, ProtoSSM sequence model over 12 windows	Aves	35% Aves	Added sequence/context signal and embedding diversity that was not just another mel CNN.
EfficientNetV2-B3 Aves SED	EfficientNetV2-B3, 10 s mel SED, XCL/pretrain warm start, WABAD rows	Aves	5% Aves	Small CNN sidecar for rank diversity against BirdNET/Perch.
Full-taxonomy raw mean	Equal raw-probability mean of three 10 s SEDs: EfficientNet-B5, ConvNeXt-Base, EfficientNetV2-B3	All 234 classes	5% Aves, 50% Amphibia/Reptilia, 25% sonotypes,	Broad fallback teacher. It replaced a weaker full-target member and

Component	Model specifics	Target used in teacher	Weight in blend	Why it was included
			100% remaining classes	carried most of the non-specialist/non-Aves coverage.
Amphibia/Reptilia EfficientNetV2-B3	EfficientNetV2-B3, 10 s mel SED, Amphibia/Reptilia-only head, BirdCLEF 2026 train-audio head transfer, non-target distractors	Amphibia/Reptilia	50% Amphibia/Reptilia, histogram anchor	Specialized herp model; less bird-biased than the full-taxonomy SEDs.
Insect parent-gate EfficientNetV2-S	EfficientNetV2-S, 10 s mel SED, Insecta head with sonotype parent-gate auxiliary loss	Sonotypes	25% sonotypes	Lower-capacity insect specialist; helped stabilize sonotype pseudo labels.
Insect parent-gate EfficientNet-B7	EfficientNet-B7, same parent-gate sonotype recipe, high drop path	Sonotypes	25% sonotypes	High-capacity sonotype specialist; useful despite noisy labels.
Insect parent-gate ConvNeXt-Tiny	ConvNeXt-Tiny, same parent-gate sonotype recipe	Sonotypes	25% sonotypes, sonotype anchor	Architecture diversity for sonotypes; avoided making the insect teacher a pure EfficientNet ensemble.

The group assembly was:

Group	Blend rule
Aves	55% BirdNET V3 Aves + 35% Perch ProtoSSM Aves + 5% full-taxonomy raw mean + 5% EfficientNetV2-B3 Aves. Non-anchor members were histogram-matched to BirdNET V3.
Amphibia/Reptilia	50% Amphibia/Reptilia EfficientNetV2-B3 + 50% full-taxonomy raw mean. The raw mean was histogram-matched to the specialist.
Sonotypes	25% full-taxonomy raw mean + 25% EfficientNetV2-S parent-gate + 25% EfficientNet-B7 parent-gate + 25% ConvNeXt-Tiny parent-gate. No histogram matching.
Remaining classes	100% full-taxonomy raw mean.

tf_efficientnetv2_b3 with hop 920 and other neck

This branch started from the same recipe but changed the head:

- BirdNET-v3-style global/frequency/time attention neck.
- MC-dropout style global head.
- hop_length=920 and smaller time masking to preserve the wider time-grid behavior.

- Head-row transfer from the old SED head into global/frequency/time heads.

It was not the strongest standalone branch, but it had useful residual signal after histogram matching. I kept it at 20% of the non-Proto mass: enough to diversify the main model, not enough to let a weaker calibration dominate.

ProtoSSM / Perch branch

The Perch branch used public Perch v2 assets but not as a raw high-weight public notebook prediction.

Pipeline:

1. Run Perch v2 to get class scores and 1536-d embeddings for every 5 s window.
2. Build site/hour prior tables from train soundscape labels.
3. Train MLP probes on PCA-compressed Perch embeddings for classes with enough positives.
4. Train a lightweight ProtoSSM over the 12-window sequence.
5. Train a ResidualSSM correction on top of the first-pass sequence predictions.
6. Apply temperature scaling, file-confidence scaling, rank-aware scaling, adaptive delta smoothing, and per-class thresholds.

For the final kernel I loaded the trained train-side bundle from an artifact rather than retraining it inside the scoring notebook. That saved enough CPU budget to keep the two SED branches in the final stack.

The ProtoSSM branch mattered much more for non-Aves than Aves, so the final weights were asymmetric: 0.10 for Aves, 0.50 for non-Aves.

Final blending

The final blend had three details that mattered:

1. Histogram matching before averaging

ProtoSSM had different marginal probability distributions. Direct averaging made the blend weight depend on calibration scale. I instead matched each class distribution to main model, preserving rank order from the branch while putting all branches on the same per-class scale.

2. Taxon-aware weights

Aves were already handled well by main model. Non-Aves needed Perch-derived sequence/context signal. This is why the final weights were:

```
Aves:      0.9 SED + 0.10 * ProtoSSM
non-Aves: 0.5 SED + 0.50 * ProtoSSM
```

Best submission would have been with even less ProtoSSM weight for aves and insects

Inference

The SED branches used OpenVINO. SED ran 20 s windows at 10 s stride and then converted model windows back to Kaggle 5 s rows. The Perch branch used the ONNX Perch export and loaded the trained ProtoSSM artifact.

The main runtime constraint was not one model; it was keeping enough diversity under the 90-minute CPU cap. This is why I avoided adding more full SED branches at the end and instead used a small but fast branch plus the cached ProtoSSM artifact.

What worked

- 20 s SED context was consistently better than treating each 5 s row independently.
- Own SED students were worth training; relying only on public Perch/ProtoSSM forks was too correlated with the public plateau.
- Sonotype-only pseudo-label sharpening was useful. Global sharpening was easier to overfit.
- Histogram matching was a good way to blend branches with incompatible calibration.
- Perch was not a trap when used as embedding/sequence signal and downweighted for Aves.
- The final taxon split was important: Aves and non-Aves wanted different branch weights.

What did not work or was too risky

- Trusting absolute CV on labelled soundscapes.
- Adding more public 0.950-style branches without real diversity.
- Giving Perch/ProtoSSM too much Aves weight because public LB liked it.
- Late rare-taxon sidecars with beautiful local metrics and no robust LB movement.
- Extra full SED branches that could not fit comfortably into the CPU budget.

Relation to other top solutions

My final solution had the same broad shape but less label work and less deployable diversity: two strong private SED branches plus one Perch sequence branch. That was enough for 14th, but the gap to the top looks like model diversity and label quality rather than one missing postprocess constant.

Acknowledgements

Thanks to the Cornell/Kaggle hosts and to the recordists whose audio made the competition possible.

The public ecosystem mattered a lot: Google Perch v2, the Perch ONNX export, Perch metadata datasets, public SED/ProtoSSM notebooks, and the many writeups from BirdCLEF 2024 and 2025. In particular, the 2024/2025 top solutions made the winning pattern clear before the 2026 leaderboard did: long-context SEDs, careful pseudo labels, temporal/file postprocessing, and inference engineering.

See you next year.